

1. How would you evaluate the security

Map listening ports and processes, check if each service is strictly necessary for the system's role, inspect version numbers for unpatched CVEs, and verify that strong authentication

2. prioritize detection accuracy, non-destructiveness, crash, support for windows-specific features, and clear remediation reporting.

3. Check if RPC interfaces allow unauthenticated queries, test for known remote execution exploits and ensure ports are blocked from untrusted networks

4. A null session allows anonymous connection, to IPC\$ without credentials.

Attackers can enumerate disclosure: usernames, password policies.

5. severity depends on the data stored and permissions granted. write/modify access, severity is critical

6. vulnerable service: flaw in the software code itself
ex: buffer, overflow.

Misconfigured services: safe software deployed incorrectly (ex: passwords, unquoted service path).

7. Measure whether the reconnaissance produced actionable attack paths (valid user lists, software versions, domain structure).

8. **Local**: Local vulnerabilities require an existing foothold and usually serve as privilege escalation.
- Remote**: It is far more critical attacker needs no prior access to breach the system.
9. Unneeded services broaden the attack surface, waste system memory and CPU.
10. These legacy files allow remote execution based solely on IP/hostname trust without password checks.
11. Review `/etc/exports` for open wildcards (*) are dangerous flags like `no-root_squash`.
12. If critical systems files (`/etc/passwd`, `/etc/sudoers`, cron jobs) are writable by regular users, the risk is critical.
13. **Weak permissions**: very easy reliable to exploit using basic shell commands; `chmod`.
- unpatched CVEs**: Harder to exploit, but can allow initial remote compromise or kernel-level control.
14. Evaluating binary protection using `checksec`. If protections are absent or bypassable.
15. System architecture, kernel version, active defensive mitigations (cf Linux, ASLR).

16. Suspect local application storage for plain text data. verify if AES encryption is applied and if encryption

17. Almost always critical. Attackers can bypass authentication. steal complete database passwords, p11.

18. **critical**: direct remote code execution, unauthenticated bypass.

low-risk: missing non-critical HTTP security headers, verbose banner displays, or self-XSS.

19. Run static analysis to find hard coded keys, weak permissions, and exported components in the manifest
dynamic analysis. (clipboard usage and data).

20. violates the principle of least privilege. If the app is breached

21. Because mobile binaries are easily decompiled, hardcode keys.

22. web: exploited centrally on the server;

immediately compromises backend

mobile: exploited on individual client devices

patching takes much longer

23. Windows: active directory and OS tools.

Linux: privilege - escalation checker

web: web proxies and injection scanners.

mobile: decompilers and runtime frameworks

All the exploit must achieve a concrete, measurable impact, and run reliably and repeatedly

SS prioritization: Rank by CVSS score and exploitability - address unauthenticated remote code.

Report structure:

executive summary for leadership, a risk/severity dashboard, detailed findings (proof of concept, impact and fix).